Module 3
Conditional Statements in Python

Agenda

1. Decision Making
2. Conditional Statements
3. Operators in Decision Making
4. Boolean Evaluation
5. Short-Circuit Evaluation
6. Nested Conditional Logic
7. Clean Conditional Code & Arrow Anti-Pattern

© Copyright Intellipaat. All rights reserved.

Decision Making

© Copyright Intellipaat. All rights reserved.

Introduction to Decision Making

Decision making is the process of **selecting an action based on one or more conditions.**

In programming, decision-making allows a program to:

- Analyze conditions
- Choose between alternatives
- Execute different code paths

Without decision-making, programs would execute every instruction sequentially **without any intelligence**.

Real-life Example



- If temperature > 30°C
  → Turn ON fan
- Else
  → Keep fan OFF

How Decision Making Works?



Input
Receive data or information

Condition Check
Evaluate the condition (True / False)

Decision
Choose the appropriate path (Yes / No)

Action
Execute the selected action or path

© Copyright Intellipaat. All rights reserved.

Why Decision Making Matters?

Decision making makes programs **interactive**, **dynamic**, and **intelligent**.

Without conditional logic:

- Programs cannot respond to user input
- Same output would occur every time
- No automation based on different situations

With decision making, programs can:

- Make smart choices
- Handle multiple scenarios
- Respond differently to different inputs

Real-Life Use Cases



ATM Machine

Check balance before withdrawal



Login System

Verify username & password



E-commerce Website

Apply discount if coupon valid



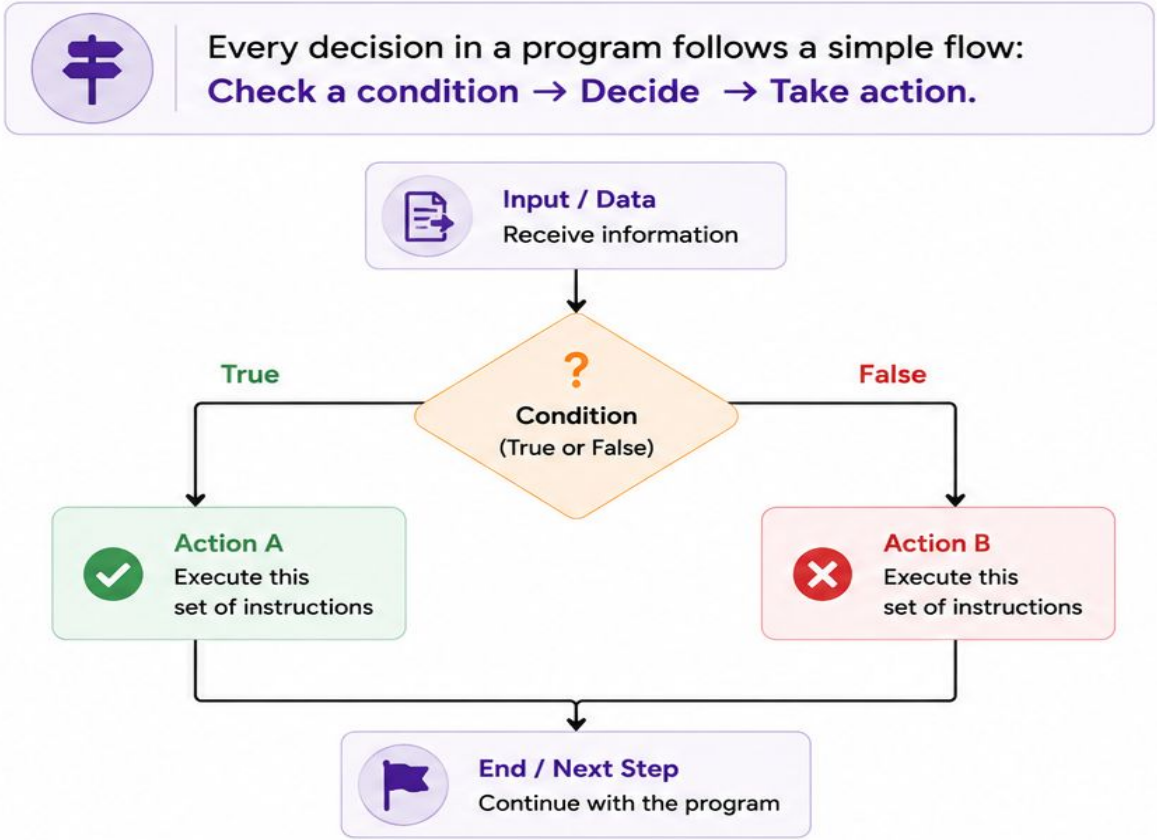
Traffic Signal System

Control vehicle movement

Key Takeaway: Decision-making enables software to behave **differently** under different conditions.

© Copyright Intellipaat. All rights reserved.

Real-Life Decision Flow



© Copyright Intellipaat. All rights reserved.

Hands-On: Decision Making

© Copyright Intellipaat. All rights reserved.

Hands-On: Decision Making

Problem Statement

A parking lot system must decide whether a vehicle can enter.

The vehicle can enter only if at least **one parking space is available**.

Assume that the number of available parking spaces is **5**.

Write a program that checks the availability and prints a **clear decision** for the driver.



Expected Output

Parking Spaces Available: 5

Vehicle can enter.



© Copyright Intellipaat. All rights reserved.

Conditional Statements

© Copyright Intellipaat. All rights reserved.

What is a Conditional Statement

A **conditional statement** helps a program make decisions.

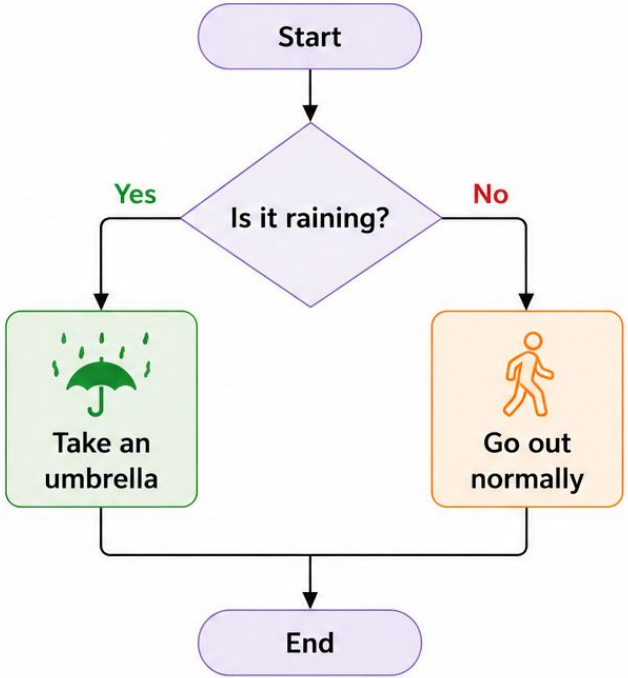
It checks a condition and chooses:

- One action if condition is **True**
- Another action if condition is **False**

Real-life Example:

If it is raining → Take an umbrella

Else → Go normally



Key Takeaway: Conditional statements allow programs to think and decide.

© Copyright Intellipaat. All rights reserved.

The if Statement

IntelliPaat

SYNTAX

```
if condition:
    # code to execute if condition is True
```

The **if** keyword starts the conditional block.

The condition is any expression that evaluates to **True** or **False**.

If the condition is **True**, the **indented** code block is executed.

EXAMPLE

```
age = 18
if age >= 18:
    print("You are eligible to vote.")
```

- **if** keyword starts the conditional block.
- Condition that evaluates to True or False.
- **Indented** code executes if condition is True.

Key Takeaway: **if** statement checks a condition and runs a block of code only when the condition is True.

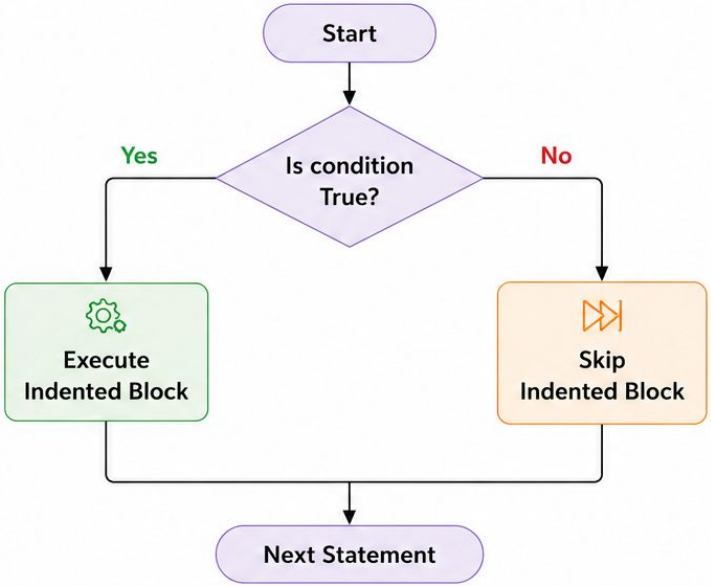
© Copyright Intellipaat. All rights reserved.

Flow of if statement

How it Works

1. **Check the Condition**
   The program evaluates the condition after **if**.
2. **If True**
   The indented block of code is executed.
3. **If False**
   The indented block is skipped and the program moves to the next line.
4. **Continue Execution**
   After executing (or skipping) the block, the program continues.

Flowchart



Key Takeaway: The **if** statement checks a condition. If it is **True**, the block **runs**; otherwise, it is **skipped**, and the program moves to the next statement.

© Copyright Intellipaat. All rights reserved.

The if-else Statement

What is if-else?

The **if-else** statement allows a program to choose between two blocks of code.

- If the condition is **True** → execute the **if** block.
- If the condition is **False** → execute the **else** block.

Real-life Example



If it rains
Take an umbrella



Else
Go out normally

Syntax

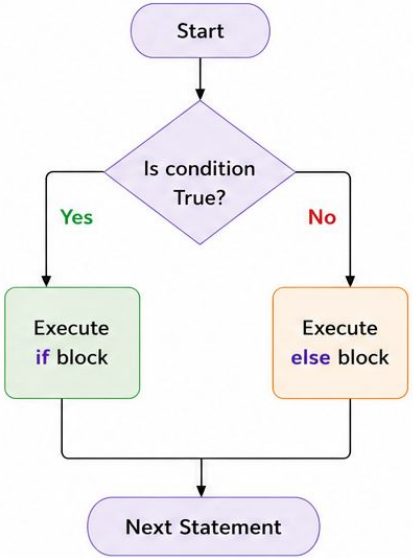
```
if condition:
    # code if condition is True
else:
    # code if condition is False
```

Example

```
age = 18
if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")
```

Only one block (**if** or **else**) is executed, not both.

Flowchart



Key Takeaway: Use **if-else** when you have two possible outcomes. It improves your program's decision-making.

© Copyright Intellipaat. All rights reserved.

Why Do We Need elif?

The Problem with only if-else

If we use only if-else, we can check only **two possibilities**.

What elif Does

elif (else if) lets us check **multiple conditions** one by one.

Real-life Analogy

Like checking your marks:
First **A grade**, else **B grade**, else **C grade**.

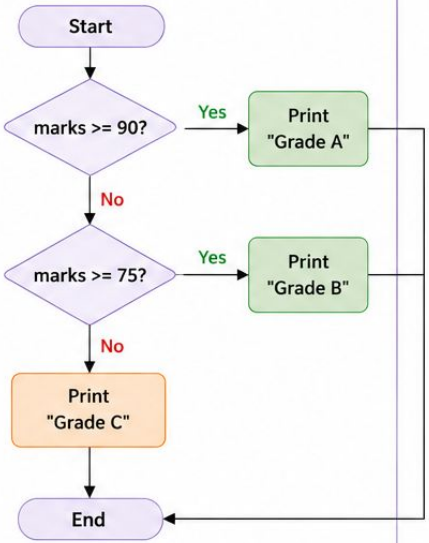
Example Code

```
marks = 85

if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
```

Example: marks = 85
Output: Grade B

Flow of Execution



Key Takeaway: Use elif when you need to check more than two conditions in sequence.

© Copyright Intellipaat. All rights reserved.

The if-elif-else Statement

What is if-elif-else?

The **if-elif-else** statement allows a program to choose one block of code from **three or more options** based on conditions.

How it Works

1. **if** — The **if** condition is checked first. If **True**, its block is executed.
2. **elif** — If **False**, each **elif** condition is checked in order.
3. **else** — If all conditions are **False**, the **else** block is executed.

</> Syntax

```
if condition1:
    # code block if condition1 is True
    ...
elif condition2:
    # code block if condition2 is True
    ...
elif condition3:
    # code block if condition3 is True
    ...
...
else:
    # code block if none of the
      conditions are True
    ...
```

Key Takeaway: Use **if-elif-else** when you have **three or more** possible outcomes. It helps in writing **clean, organized**, and **easy-to-read** code.

© Copyright Intellipaat. All rights reserved.

Flow of if-elif-else Statement

IntelliPaat

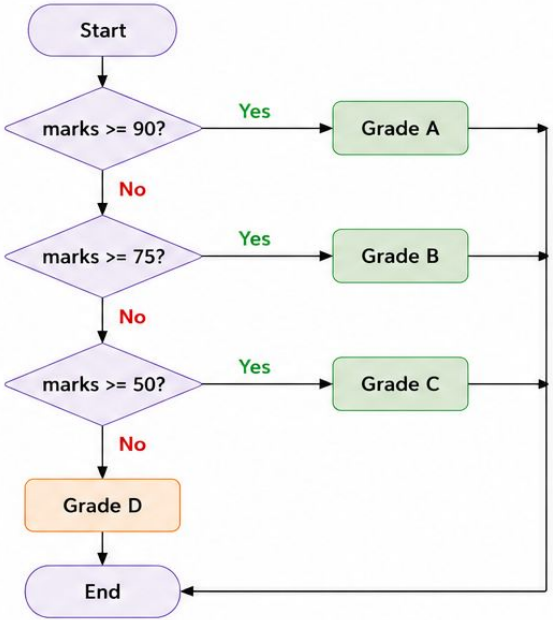
Example

```
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Grade D")
```

Output

Example: marks = 85
Output: Grade B

Flowchart



Key Takeaway: The first True condition's block runs.
If none are True, the else block runs.

© Copyright Intellipaat. All rights reserved.

Hands-On: Conditional Statements

© Copyright Intellipaat. All rights reserved.

Hands-On: Conditional Statements

Problem Statement

A cinema offers ticket discounts based on the customer's age.

- If the age is **60 or above**, the ticket costs **₹120**.
- If the age is **between 13 and 59**, the ticket costs **₹200**.
- If the age is **below 13**, the ticket costs **₹100**.

Assume the customer's age is **10**. Use an **`if-elif-else`** statement to determine and print the applicable ticket price.



Expected Output

Age: 10

Ticket Price: ₹100



© Copyright Intellipaat. All rights reserved.

Operators in Decision Making

© Copyright Intellipaat. All rights reserved.

What are Operators?

What are Operators?

Operators are **symbols** that tell Python what **action** to perform on values.

They help us:

- Add or compare values
- Check conditions
- Make decisions in programs

Real-Life Example

Think of operators as **action symbols**:



`+` → Add

`>` → Check which is bigger

`==` → Check equality

Examples in Python

+

```
5 + 3
```

→ 8 — Addition

>

```
10 > 7
```

→ True — Comparison

==

```
4 == 4
```

→ True — Equality Check

Key Takeaway: Operators tell Python what **action** to take.

© Copyright Intellipaat. All rights reserved.

Types of Operators used in Decision Making

IntelliPaat

What to know?

Python provides different types of operators that help us perform **comparisons**, **combine conditions**, and **test relationships**.



Comparison Operators

Used to compare two values and return True or False.

Examples

`==` `!=` `>` `<` `>=` `<=`

Example in Python

```
10 > 5    → True
```

Logical Operators

Used to combine two or more conditions and return True or False.

Examples

`and` `or` `not`

Example in Python

```
(5 > 3) and (2 < 4) → True
```

Membership Operators

Used to test whether a value is present in a sequence.

Examples

`in` `not in`

Example in Python

```
'a' in 'apple'   → True
```

Identity Operators

Used to check whether two variables refer to the same object.

Examples

`is` `is not`

Example in Python

```
x is y    → True
```

Key Takeaway: These operators help Python make smart decisions and control the flow of our programs.

© Copyright Intellipaat. All rights reserved.

Comparison Operators

What are Comparison Operators?

Comparison operators are used to compare two values.
They always return **True** or **False**.



==
Equal to
Returns True if both values are equal.
Example: `5 == 5` → True

!=
Not equal to
Returns True if both values are **not** equal.
Example: `5 != 8` → True

>
Greater than
Returns True if left value is **greater** than right value.
Example: `10 > 7` → True

<
Less than
Returns True if left value is less than right value.
Example: `3 < 9` → True

>=
Greater than or equal to
Returns True if left value is greater than or equal to right value.
Example: `6 >= 6` → True

<=
Less than or equal to
Returns True if left value is less than or equal to right value.
Example: `4 <= 7` → True

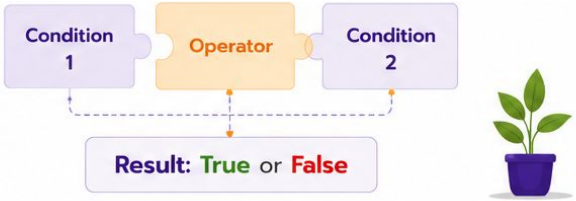
Key Takeaway: Comparison operators help Python compare values and return **True** or **False**.

© Copyright Intellipaat. All rights reserved.

Logical Operators

What are Logical Operators?

Logical operators are used to combine two or more conditions and return True or False.



&& and

Returns True only if both conditions are True.

Example in Python

```
(5 > 2) and (10 < 20)  →  True
```

Truth Table

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

|| or

Returns True if at least one of the conditions is True.

Example in Python

```
(5 > 2) or (10 > 20)  →  True
```

Truth Table

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

! not

Reverses the result. Returns True if the condition is False, and vice versa.

Example in Python

```
not (5 > 2)  →  False
```

Truth Table

A	not A
True	False
False	True

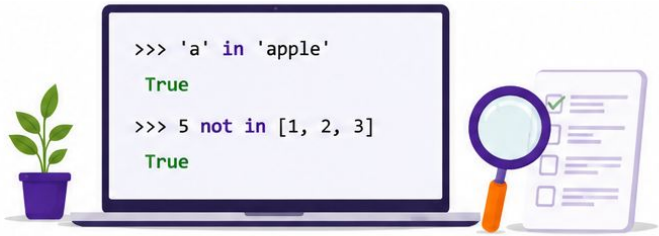
Key Takeaway: Logical operators help combine or modify conditions and return True or False.

© Copyright Intellipaat. All rights reserved.

Membership Operators

What are Membership Operators?

Membership operators are used to test whether a value is present in a **sequence** (like string, list, tuple, set, or dictionary).



in Operator

Returns True if the value is found in the sequence.

Examples in Python

abc	`'a' in 'apple'`	→	True
[ ]	`5 in [1, 2, 3, 4, 5]`	→	True
key:value	`'name' in {'name': 'John', 'age': 20}`	→	True

Works with: strings, lists, tuples, sets, dictionaries

not in Operator

Returns True if the value is not found in the sequence.

Examples in Python

abc	`'z' not in 'apple'`	→	True
[ ]	`10 not in [1, 2, 3, 4, 5]`	→	True
key:value	`'city' not in {'name': 'John', 'age': 20}`	→	True

Useful when you want to check absence of a value.

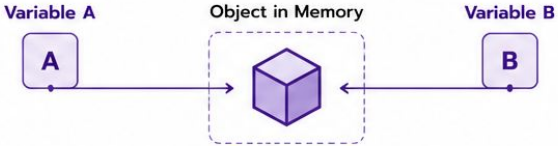
Key Takeaway: Membership operators help you check whether a value exists in a sequence or not.

© Copyright Intellipaat. All rights reserved.

Identity Operators

What are Identity Operators?

Identity operators are used to check whether two variables refer to the same object in memory (**same identity**).



If both point to the same object → True



is Operator

Returns True if both variables refer to the **same object**.

Examples in Python

`a = [1, 2, 3]`	`b = a`	→	True
`x = 10`	`y = 10`	→	True
`p = [1, 2]`	`q = [1, 2]`	→	False

Checks if both variables point to the **exact same object**.

is not Operator

Returns True if both variables **do not** refer to the **same object**.

Examples in Python

`a = [1, 2, 3]`	`b = [1, 2, 3]`	→	True
`x = 10`	`y = 20`	→	True
`p = [1, 2]`	`q = p`	→	False

Checks if both variables point to **different objects**.

Key Takeaway: Identity operators help you check whether two variables refer to the same object in memory.

© Copyright Intellipaat. All rights reserved.

Other Operators in Python

Beyond the Basics

Python provides some additional operators that help simplify code and work at a lower level.

Arithmetic Operators

Perform basic mathematical operations on values.

Examples

Operator	Meaning	Example	Eesult
+	Addition	5 + 2	7
-	Subtraction	5 - 2	3
*	Multiplication	5 * 2	10
/	Division	5 / 2	2.5
%	Modulus	5 % 2	1
**	Exponentiation	5 ** 2	25
//	Floor Division	5 // 2	2

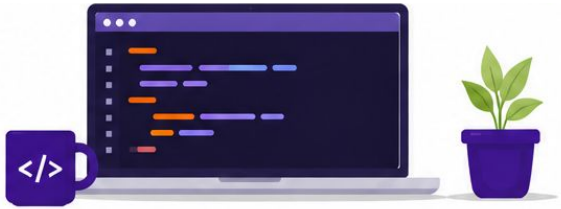
Use case: Used for performing standard mathematical calculations.

© Copyright Intellipaat. All rights reserved.

Other Operators in Python

Beyond the Basics

Python provides some additional operators that help simplify code and work at a lower level.



Arithmetic Operators

Perform basic mathematical operations on values.

Examples

Operator	Meaning	Example	Eesult
+	Addition	5 + 2	7
-	Subtraction	5 - 2	3
*	Multiplication	5 * 2	10
/	Division	5 / 2	2.5
%	Modulus	5 % 2	1
**	Exponentiation	5 ** 2	25
//	Floor Division	5 // 2	2

Use case: Used for performing standard mathematical calculations.

Assignment Operators

Assign values to variables.
The basic form is =.

Examples

Operator	Meaning	Example	Same As
=	Assign	x = 5	x = 5
+=	Add and assign	x += 3	x = x + 3
-=	Subtract and assign	x -= 3	x = x - 3
*=	Multiply and assign	x *= 3	x = x * 3
/=	Divide and assign	x /= 3	x = x / 3
%=	Modulus and assign	x %= 3	x = x % 3
**=	Power and assign	x **= 3	x = x ** 3
//=	Floor divide and assign	x //= 3	x = x // 3

Use case: Shortens code and updates values efficiently.

Bitwise Operators

Work at the bit level on integer values.

Examples

Operator	Meaning	Example
&	Bitwise AND	5 & 3
|	Bitwise OR	5 | 3
^	Bitwise XOR	5 ^ 3
~	Bitwise NOT	~5
<<	Left shift	5 << 1
>>	Right shift	5 >> 1

Use case: Useful in low-level programming, optimizations, and bit manipulation.

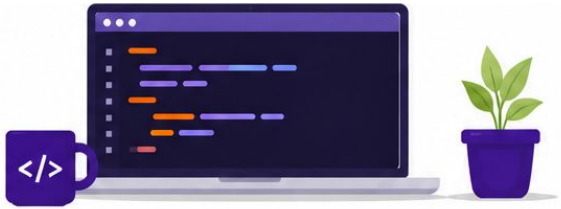
Key Takeaway: Arithmetic, assignment, and bitwise operators help write cleaner, shorter, and more efficient code.

© Copyright Intellipaat. All rights reserved.

Other Operators in Python

Beyond the Basics

Python provides some additional operators that help simplify code and work at a lower level.



Arithmetic Operators

Perform basic mathematical operations on values.

Examples

Operator	Meaning	Example	Eesult
+	Addition	5 + 2	7
-	Subtraction	5 - 2	3
*	Multiplication	5 * 2	10
/	Division	5 / 2	2.5
%	Modulus	5 % 2	1
**	Exponentiation	5 ** 2	25
//	Floor Division	5 // 2	2

Use case: Used for performing standard mathematical calculations.

Assignment Operators

Assign values to variables.
The basic form is =.

Examples

Operator	Meaning	Example	Same As
=	Assign	x = 5	x = 5
+=	Add and assign	x += 3	x = x + 3
-=	Subtract and assign	x -= 3	x = x - 3
*=	Multiply and assign	x *= 3	x = x * 3
/=	Divide and assign	x /= 3	x = x / 3
%=	Modulus and assign	x %= 3	x = x % 3
**=	Power and assign	x **= 3	x = x ** 3
//=	Floor divide and assign	x //= 3	x = x // 3

Use case: Shortens code and updates values efficiently.

Bitwise Operators

Work at the bit level on integer values.

Examples

Operator	Meaning	Example
&	Bitwise AND	5 & 3
|	Bitwise OR	5 | 3
^	Bitwise XOR	5 ^ 3
~	Bitwise NOT	~5
<<	Left shift	5 << 1
>>	Right shift	5 >> 1

Use case: Useful in low-level programming, optimizations, and bit manipulation.

Key Takeaway: Arithmetic, assignment, and bitwise operators help write cleaner, shorter, and more efficient code.

© Copyright Intellipaat. All rights reserved.

Hands-On: Operators in Decision Making

© Copyright Intellipaat. All rights reserved.

Hands-On: Operators in Decision Making

Problem Statement

An employee is eligible for a performance bonus if the employee has completed at least **90%** of the assigned tasks and has received manager approval.

- Create a list of assigned skills containing **Python** and **SQL**.
- Check whether **SQL** is present in the skills list.
- Also create another variable that refers to the same list and use an **identity operator** to verifythat both variables refer to the same object.

Use **task completion = 92** and set **manager approval = True**.



Expected Output

Task Completion Requirement Met: True

Manager Approval: True

SQL Skill Present: True

Same Skills List: True



© Copyright Intellipaat. All rights reserved.

Boolean Evaluation

© Copyright Intellipaat. All rights reserved.

Introduction to Boolean Evaluation

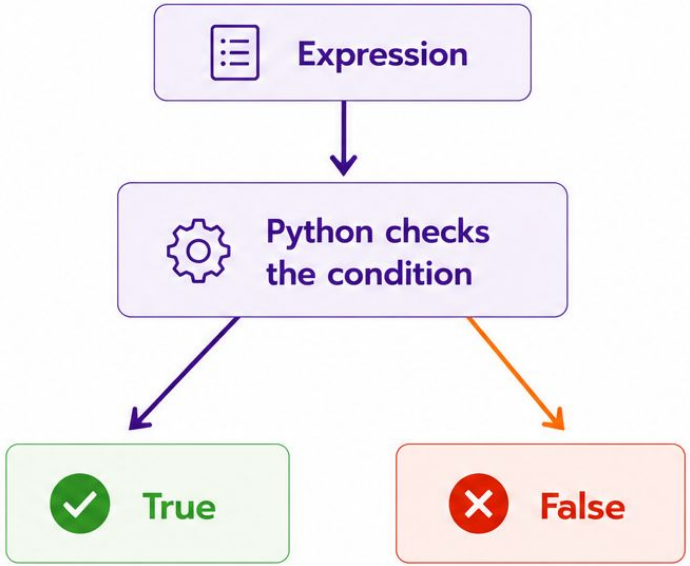
Boolean Evaluation

Python checks whether a value or expression is:

- True
- False

Example

```
10 > 5  →  True
```



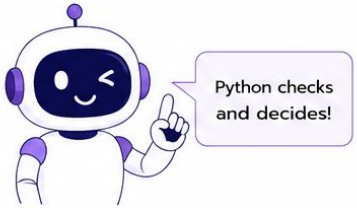
Key Takeaway: Every condition in Python becomes True or False.

© Copyright Intellipaat. All rights reserved.

Truthy vs Falsy Values

Every value in Python behaves as either **Truthy** or **Falsy**.





Truthy Values

These values are considered **True**.

123	`10, -5, 3.14`	Non-zero numbers
”	`'Hello', 'Python'`	Non-empty strings
☰	`[1, 2, 3], (1, 2)`	Non-empty collections (lists, tuples, dicts, sets, etc.)

Most objects in Python are truthy by default.

Falsy Values

These values are considered **False**.

0	`0, 0.0`	Zero of any numeric type
”	`'' or ""`	Empty string
[]	`[], (), {}, set()`	Empty list, tuple, dictionary, set
N	`None`	None value
False	`False`	Boolean False

These values are treated as **False** in conditions.

Key Takeaway: Python evaluates values as either **True** or **False** during decision making.

© Copyright Intellipaat. All rights reserved.

Boolean Evaluation in Action

Python evaluates values and expressions and decides whether they are True or False.

Expression / Value	Result
`bool(10)` →	True
`bool(0)` →	False
`bool("")` →	False
`bool([1, 2, 3])` →	True

How it works?

Expression or Value → Python evaluates the expression/value

- **True** — Condition satisfied. (Block runs)
- **False** — Condition not satisfied. (Block skipped)

Example in Use

```
if 10 > 5:
    print("Greater")
```

→ True (Condition satisfied) Block executes.

```
if []:
    print("Not Empty")
```

→ False (Condition not satisfied) Block skipped.

Key Takeaway: Python converts every expression or value into either True or False so that programs can make decisions.

© Copyright Intellipaat. All rights reserved.

Hands-On: Boolean Evaluation

© Copyright Intellipaat. All rights reserved.

Hands-On: Boolean Evaluation

Problem Statement

Create variables for the following shopping cart details:

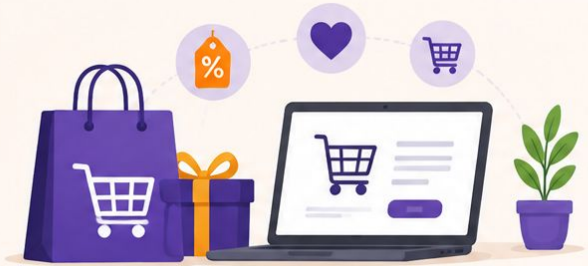
- **items_in_cart** as an empty list
- **store_name** as **"Tech Store"**
- **discount** as **0**
- **wishlist** as a list containing **"Laptop"**

Use **bool()** to check whether each value is truthy or falsy, then print the result for every item.



Expected Output

- Items in Cart: **False**
- Store Name Available: **True**
- Discount Applied: **False**
- Wishlist Present: **True**



© Copyright Intellipaat. All rights reserved.

Short-Circuit Evaluation

© Copyright Intellipaat. All rights reserved.

Short-Circuit Evaluation

Python sometimes stops checking conditions early when the result is already known.
This is called **Short-Circuit Evaluation**.

What is it?

In logical expressions, Python does not always evaluate the second condition.

If the result can be determined from the first condition it stops right there.

&& — **and** If the first condition is **False**, the whole expression is **False**. Python does not check the rest.

|| — **or** If the first condition is **True**, the whole expression is **True**. Python does not check the rest.

How it works?

1 For **and**



First condition is **False** → Result is already **False** →
Python stops and does not check the second condition.

2 For **or**



First condition is **True** → Result is already **True** →
Python stops and does not check the second condition.

Key Takeaway: Python avoids unnecessary checks to save time and resources.

© Copyright Intellipaat. All rights reserved.

Short-Circuit with and

IntelliPaat

&

The **and** operator returns **True** only if both conditions are **True**.

Rule:

- If the first condition is **False**, Python stops immediately.
- The second condition is **not checked**.

Example

```
x = 5
print(x > 10 and x / 0 > 1)
```

Output:

```
False
```

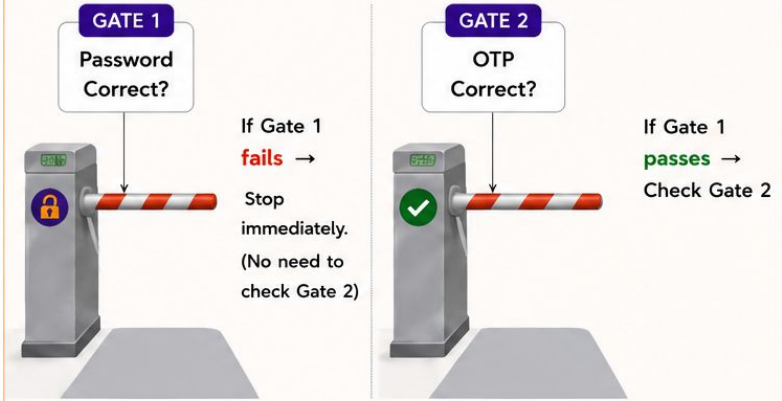
Step-by-step:

1. `x > 10` → **False**
2. Python **stops** here.
3. `x / 0` never executes.

Important: No division error occurs!

Real-Life Example: Online Exam Login

Condition: Password correct **AND** OTP correct



Key Takeaway:

With **and**, Python stops at the first **False** because the overall result cannot become **True** anymore.

© Copyright Intellipaat. All rights reserved.

Short-Circuit with or

IntelliPaat

OR

The or operator returns True if at least one condition is True.

Rule:

- If the first condition is True, Python stops immediately.
- The second condition is not checked.

Example

```
x = 15
print(x > 10 or x / 0 > 1)
```

Step-by-step:

1. x > 10 → True
2. Python stops here.
3. x / 0 never executes.

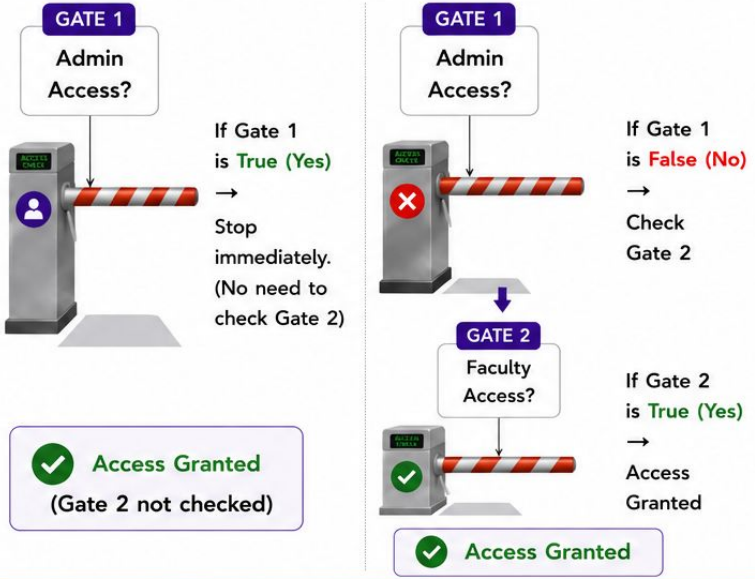
Output:

```
True
```

Important: No division error occurs!

Real-Life Example: Online Exam Login

Condition: Admin Access OR Faculty Access



Key Takeaway:

With or, Python stops at the first True because the overall result cannot become False anymore.

© Copyright Intellipaat. All rights reserved.

Real World Optimization Example

Short-circuit evaluation helps Python **stop early**, **saving time** and avoiding unnecessary checks.
It lets Python do only what is necessary.

SHORT-CIRCUIT WITH AND



Real-World Example:
Online Payment Verification

Payment allowed only if:
Card is valid **AND** Bank OTP is correct.

How it works

- Step 1: Is card valid? — No → STOP
- Yes
- Step 2: Is OTP correct? — No → STOP
- Yes
- Payment Success

Code Example

```
card_valid and otp_correct
```

- If `card_valid` is **False**, Python stops immediately.
- `otp_correct` is **not checked.**

Benefit

Fail fast.
Skip expensive checks when the first step fails.

SHORT-CIRCUIT WITH OR



Real-World Example:
Website Login Access

Access granted if:
User is Admin **OR** User is Faculty.

How it works

- Step 1: Is user Admin? — Yes → STOP
- No
- Step 2: Is user Faculty? — Yes → STOP
- No
- Access Denied

Code Example

```
is_admin or is_faculty
```

- If `is_admin` is **True**, Python stops immediately.
- `is_faculty` is **not checked.**

Benefit

Succeed fast.
Skip further checks once access is confirmed.

© Copyright Intellipaat. All rights reserved.

Hands-On: Short-Circuit Evaluation

© Copyright Intellipaat. All rights reserved.

Hands-On: Short-Circuit Evaluation

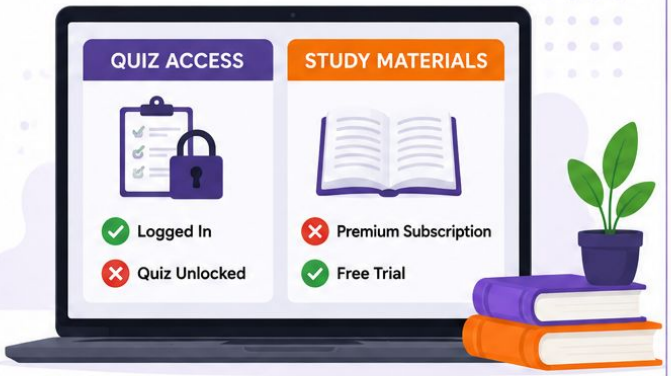
Problem Statement

An online learning platform allows a student to start a quiz only if the student is **logged in** and the **quiz is unlocked**.

It allows access to study materials if the student has either a **premium subscription** or a **free trial**.

Assume the student is **logged in**, the quiz is **not unlocked**, the student **does not** have a premium subscription, and **does** have a free trial.

Use **and** for the quiz access check **and or** for the study material access check. Print the final decision for both requests.



Expected Output

- Quiz Access Check: False
- Study Material Access Check: True
- Quiz Access: Denied
- Study Material Access: Allowed



© Copyright Intellipaat. All rights reserved.

Nested Conditional Logic

© Copyright Intellipaat. All rights reserved.

Introduction to Nested Conditionals

What are Nested Conditional Blocks?

A nested condition means placing one `if` statement inside another `if` statement.

How Python Checks:

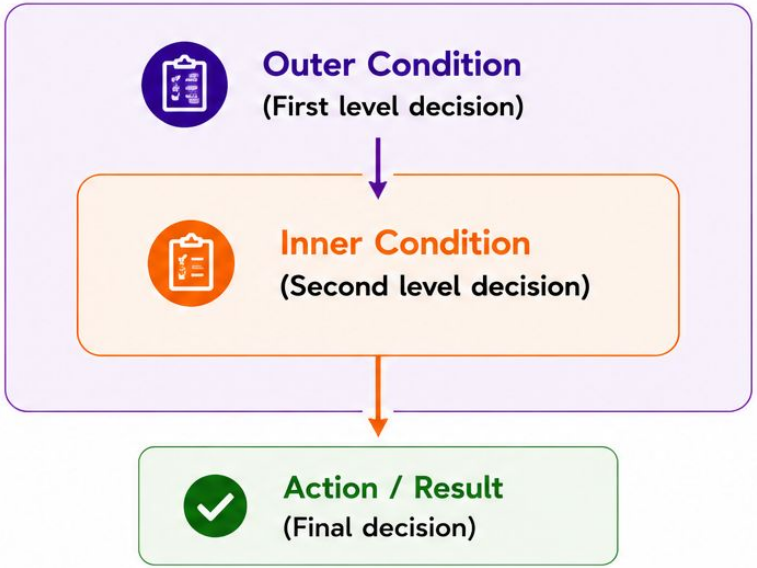
1. Checks the **outer condition** first.
2. Checks the **inner condition** next, only if the outer condition is **True**.

Real-World Hint:

To enter the exam hall:

- Valid Admit Card?
- If yes → Check ID card

Visual Representation



Key Takeaway: Nested conditions help Python make **multi-level decisions** based on multiple checks.

© Copyright Intellipaat. All rights reserved.

How Nested Conditionals Work

Flow of Nested Evaluation

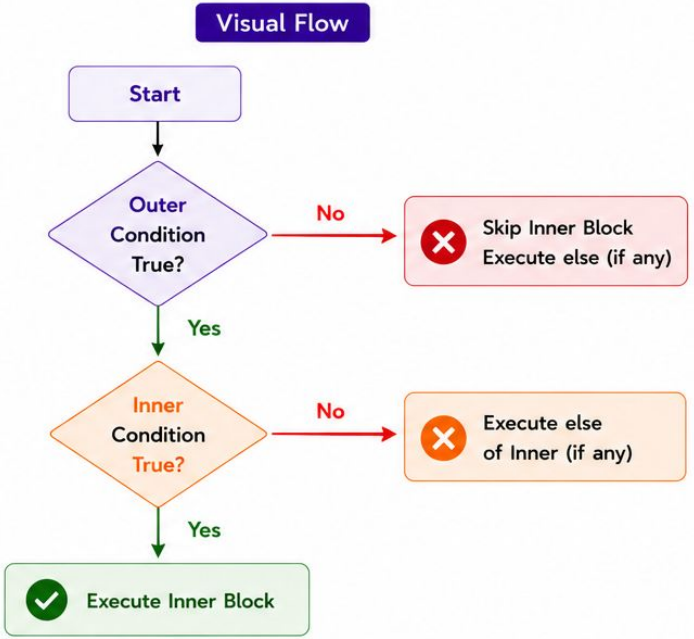
Python evaluates conditions from the outer level to the inner level.

1 Check Outer Condition
Python first checks the outer if condition.

2 If True, Check Inner Condition
Only when the outer condition is True, Python checks the inner if condition.

3 Take Action
Based on the results, Python executes the appropriate block.

Python moves downward only when the current condition is True.



Key Takeaway:

Python first checks the outer condition.
The inner condition is checked only if the outer condition is True.

© Copyright Intellipaat. All rights reserved.

Real-World Example

Nested conditional blocks are used when one decision depends on the result of another decision.
In an ATM, the system checks conditions in sequence.

Scenario



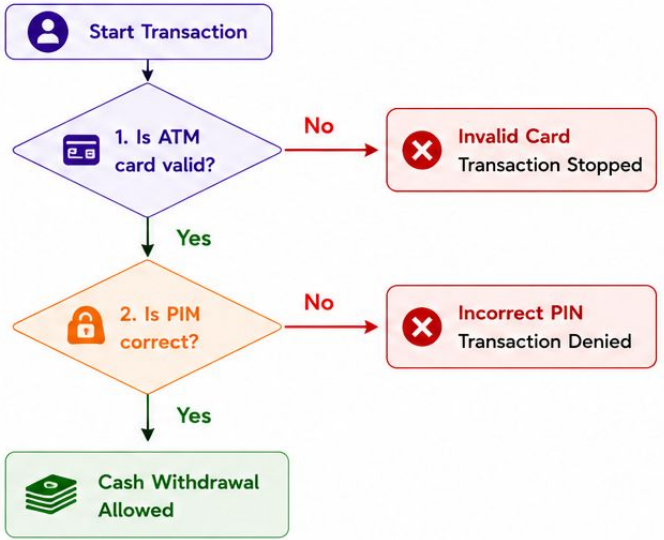
To withdraw cash from an ATM:

1. Check if the ATM card is valid.
   - ✅ Yes → Go to next step
   - ❌ No → Transaction stopped
2. Check if the PIN is correct.
   - ✅ Yes → Cash withdrawal allowed
   - ❌ No → Transaction denied

Python Code

```
if card_valid:
    if pin_correct:
        print("Cash Withdrawal Successful ✅")
    else:
        print("Incorrect PIN ❌")
else:
    print("Invalid Card ❌")
```

How it Works (Visual Flow)



Key Takeaway:

Nested conditional blocks help Python make decisions in multiple steps — first the **outer condition**, then the **inner condition**.

© Copyright Intellipaat. All rights reserved.

Common Mistakes in Nested Logic

Nested conditionals help solve multi-level decisions, but improper usage can make code **hard to read**, **difficult to debug**, and **error-prone**.

1 Too Many Nested Levels

Deep nesting creates arrow-shaped code that is **hard to read** and follow.

Bad Example

```
if logged_in:
    if has_subscription:
        if payment_verified:
            if profile_complete:
                print("Access Granted")
```

Too much indentation = Arrow-shaped code

Why it's a problem

- Hard to track the flow
- Difficult to modify
- Increases chance of logical errors

Aim for fewer levels and use **early exits** when possible.

2 Unnecessary Nesting

Nesting when not needed makes code longer and harder to read.

Bad Example

```
if age >= 18:
    if has_id:
        print("Allowed")
```

Extra nesting for simple logic

Better Approach

```
if age >= 18 and has_id:
    print("Allowed")
```

Why it's a problem

- Extra indentation
- Makes logic look more complex than it is
- Less readable

Combine conditions when they are **independent** and simple.

3 Repeating Conditions

Checking the same condition again leads to **redundant** and confusing code.

Bad Example

```
if marks >= 40:
    if marks >= 40:
        print("Pass")
```

Same condition checked twice

Better Approach

```
if marks >= 40:
    print("Pass")
```

Why it's a problem

- Redundant checks
- Wastes processing
- Can lead to maintenance issues

Check each condition once.
Keep logic **simple** and **meaningful**.

© Copyright Intellipaat. All rights reserved.

Hands-On: Nested Conditional Logic

© Copyright Intellipaat. All rights reserved.

Hands-On: Nested Conditional Logic

Problem Statement

A customer can complete an online purchase only after passing several checks in order.

- First, verify that the customer is **logged in**.
- Only if logged in, check whether the **shopping cart is not empty**.
- Only if the cart contains items, check whether the **payment** was **successful**.

Assume the customer is **logged in**, the cart **contains items**, and the payment is **successful**.

Use **nested conditional statements** and print the purchase status.



Expected Output

Login Status: Verified

Cart Status: Verified

Purchase Status: Successful



© Copyright Intellipaat. All rights reserved.

Clean Conditional Code & Arrow Anti-Pattern

© Copyright Intellipaat. All rights reserved.

What is the Arrow Anti-Pattern?

The Arrow Anti-Pattern is a code structure formed by **deep nesting** of **if** statements.
It looks like a **right-pointing arrow** → increasing indentation with each level.
This pattern makes code **hard to read**, **understand**, and **maintain**.

1. What It Looks Like

Deep nesting creates an **"arrow-shaped"** code where each inner block is indented more.

```
if user_logged_in:
    if has_subscription:
        if payment_successful:
            if profile_complete:
                if email_verified:
                    print("Access Granted")
```

As we go deeper, the code moves to the right like an **arrow**.

2. Why It's a Problem

Hard to Read
Too much indentation makes it difficult to scan and follow.

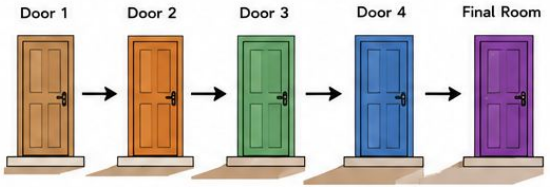
Hard to Understand
It's not easy to see the overall logic and conditions.

Hard to Debug
Finding the exact issue becomes time-consuming.

Hard to Maintain
Adding or changing logic increases the risk of errors.

3. Real-World Analogy

Imagine a set of locked doors where you need to open one door after another to reach the final room.



You must pass through every door in order.
One locked door blocks everything that follows!

© Copyright Intellipaat. All rights reserved.

How to Avoid the Arrow Anti-Pattern

IntelliPaat

1 Combine Related Conditions

Combine multiple conditions that are required together into a single if statement.

Before (Deep Nesting)

```
if user.is_active:
    if user.is_verified:
        print("Access Granted")
```

After (Better)

```
if user.is_active and user.is_verified:
    print("Access Granted")
```

Why it's better

- Fewer nested blocks
- Cleaner and easier to read
- Logic is clear and concise

2 Use Early Exit (Guard Clauses)

Handle failure or invalid cases early to avoid unnecessary nesting.

Before (Deep Nesting)

```
if order:
    if order.is_paid:
        if order.is_in_stock:
            process_order(order)
        else:
            print("Out of stock")
    else:
        print("Payment required")
else:
    print("Invalid order")
```

After (Better)

```
if not order:
    print("Invalid order")
elif not order.is_paid:
    print("Payment required")
elif not order.is_in_stock:
    print("Out of stock")
else:
    process_order(order)
```

Why it's better

- Handles failure cases early
- Avoids deep nesting
- Easier to follow the flow

© Copyright Intellipaat. All rights reserved.

CONTINUE....

3 Split Complex Logic

Break complex conditions into **meaningful** variables or smaller parts.

Before (Hard to Read)

```
if (user.role == "admin" or user.role == "moderator") and
   user.is_active and (user.signup_days > 30 or
   user.has_subscription):
     print("Access Granted")
```

After (Better)

```
is_authorized_role = user.role in ["admin", "moderator"]
has_experience = user.signup_days > 30
has_subscription = user.has_subscription

if is_authorized_role and user.is_active and \
  (has_experience or has_subscription):
     print("Access Granted")
```

Why it's better

- Improves readability
- Easier to understand each part
- Easier to test and debug

Key Takeaway:

Prefer flat, readable logic over deeply nested conditions.
Simpler code is **better code**.

General Tips

Think: Can I combine these **conditions**?

Return **early** when something is not valid.

Break big conditions into small, **named parts.**

Add **comments** to explain complex decisions.

© Copyright Intellipaat. All rights reserved.

Hands-On: Clean Conditional Code and Arrow Anti-Pattern

© Copyright Intellipaat. All rights reserved.

Hands-On: Clean Conditional Code and Arrow Anti-Pattern

Problem Statement

A university portal must decide whether to approve a student's exam hall ticket.

- Check the conditions in a clear order: the student must be **registered**, the exam fee must be **paid**, and the attendance must be **at least 75%**.
- Avoid placing each check inside another deeply nested `if` block.
- Instead, use clean `if-elif-else` conditions and print only one final approval message.

Assume the student is **registered**, the exam fee is **paid**, and the attendance is **82%**.



Expected Output

Hall Ticket Status: Approved



© Copyright Intellipaat. All rights reserved.

Thanks!



080-4524-9465



support@intellipaat.com

© Copyright Intellipaat. All rights reserved.